

Week 7 Guided Path: Contract and Swap

COMSC-1053 | Reasoning Odyssey Gate

The goal. Your program already depends on other objects. This week you will find one real dependency, write down its promise as an explicit abstract base class (ABC), plug in two different collaborators, and prove the caller can work with either one. You are not building a second project. You are making a small, honest improvement to the world you built in Weeks 4–6.

What you will make

1. An **ABC** with one abstract operation that states the collaborator’s promise.
2. Two concrete collaborators that inherit from the ABC, preserve meaningful state, and behave differently.
3. One caller that depends on the promise instead of branching on concrete types.
4. One focused `unittest` that uses the same caller and request with both collaborators.
5. A short World Bible update explaining the change, evidence, and remaining debt.

First, see the complete example

The numbered Frontier Settlement lesson is one guided example. It is not a template to copy. Its `ExpeditionPlanner` asks a `SupplySource` for a `SupplyQuote`. A `SettlementWarehouse` and a `TradingPost` both provide quotes, but their rules differ: the trading post protects a reserve.

Step	File	Question it answers
1	<code>step_01_values.py</code>	What values and errors cross this boundary?
2	<code>step_02_contracts.py</code>	What must every supply source promise?
3	<code>step_03_validation.py</code>	What inputs are valid before delegation?
4	<code>step_04_warehouse.py</code>	How can one source keep meaningful state?
5	<code>step_05_trading_post.py</code>	How can another source follow a different rule?
6	<code>step_06_planner.py</code>	What work belongs to the caller boundary?
7	<code>step_07_tests.py</code>	What evidence proves the shared promise?
8	<code>step_08_demo.py</code>	What does a visible swap look like?
9	<code>step_09_world_bible.py</code>	What changed and what debt remains?

Run it from the Week 7 directory:

```
python class_example.py test
python class_example.py demo
python class_example.py bible
```

The first command runs 34 tests. The test file is deliberately named `step_07_tests.py`, so use this entry point rather than generic test discovery at the directory root.

The idea in plain language

An explicit contract tells the caller what it may trust. In the example, the caller may trust that a supply source can answer `quote_supply(...)` with a valid `SupplyQuote`. It may *not* assume the source is a warehouse, a trading post, or any future kind of source.

The caller boundary is the moment `ExpeditionPlanner` hands a request to the collaborator and accepts a result back. A good boundary does three jobs:

- validates request data before it crosses out;
- checks the returned value still satisfies the promise; and
- adds useful context if the collaborator fails.

This is why the caller has no warehouse-specific or trading-post-specific branch. Swapping a collaborator changes behavior without rewriting the caller.

Build your own version

1. Look back at your Weeks 4–6 world. Find a dependency that already exists in the program. Ask: What does my caller need from this object?
2. State that need as a narrow promise. Prefer one operation with a meaningful return value over a broad interface.
3. Create an ABC and mark the operation with `@abstractmethod`.
4. Create two concrete subclasses. Give each its own state and rule so a substitution changes behavior for a real reason.
5. Update the caller to receive the collaborator and use only the contract.
6. Write the focused swap test. Use the same caller operation and input for each collaborator; assert only what the contract promises.
7. Run the tests, demonstrate the result, and record the design decision in your World Bible.

A focused test looks for a promise, not a sentence

A brittle test checks a warehouse’s exact wording. A contract test checks the result that every valid collaborator promises. The key shape is:

```
for source in [warehouse, trading_post]:
    planner = ExpeditionPlanner(source)
    decision = planner.evaluate_supply_request(...)
    self.assertIsInstance(decision, SupplyDecision)
```

Add collaborator-specific tests separately when you need to prove a particular rule, such as the trading post’s reserve. That keeps the shared contract test honest and keeps your failures easy to understand.

Your graded discussion post (40 points)

Your post is a shared learning resource: classmates can see it. Link your repository or attach the code, then write a concise explanation with:

Evidence	Points	What a reader should find
Real contract and two collaborators	15	An ABC, abstract operation, and two stateful implementations with genuinely different behavior.
Focused swap test	10	The same caller and request work with both collaborators; the assertion matches the shared promise.
Caller-boundary explanation	10	Plain-language explanation of validation, delegation, returned-value checking, and failure context.
Demonstration and World Bible	5	A runnable result plus what changed, evidence used, and remaining debt.

Then reply constructively to at least two classmates. Refer to a specific design or test, ask a useful question, or name an idea you can carry into your own program. Do not post private information about yourself or anyone else.

Optional reference worlds

The folders beginning `contract_swap_example` and the `week_07_contract_and_swap` folder are optional, self-contained illustrations in other settings. Read one only if it helps. They are neither extra assignments nor code to copy. The numbered Frontier Settlement path is the one complete walkthrough.

A strong Week 7 submission is small, specific, tested, and honest about what it still does not solve.